

Esta página ha sido traducida del inglés por la comunidad. Aprende más y únete a la comunidad de MDN Web Docs .

[View in English](#) [Always switch to English](#)

[View in English](#) [Always switch to English](#)

Primeros pasos con HTML

- Overview: Estructurando la web con HTML

- Siguiente

En este artículo, cubrimos los conceptos básicos de HTML. Para empezar, este artículo define elementos, atributos y todos los demás términos importantes que puedas haber escuchado. También explica dónde encajan en HTML. Aprenderás cómo se estructuran los elementos HTML, cómo se estructura una página HTML típica y otras características importantes del lenguaje básico. ¡En el camino, también tendrás la oportunidad de jugar con HTML!

¿Qué es HTML?

HTML (HyperText Markup Language, por sus siglas en inglés) es un lenguaje de marcado que indica a los navegadores web cómo estructurar las páginas web que visita. Puede ser tan complicado o tan simple como el desarrollador web quiera que sea. HTML consiste en una serie de elementos , que utiliza para encerrar, envolver o marcar diferentes partes del contenido para que aparezca o actúe de cierta manera. Las etiquetas pueden convertir el contenido en un hipervínculo para conectarse a otra página, poner palabras en cursiva, etc. Por ejemplo, considere la siguiente línea de texto:

Mi gato es muy gruñón

Si quisiéramos que el texto se mantuviera solo, podríamos especificar que es un párrafo encerrándolo en un elemento de párrafo (`<p>`):

`<p>`

Mi gato es muy gruñón

Nota: Las etiquetas en HTML no distinguen entre mayúsculas y minúsculas. Esto significa que se pueden escribir en mayúsculas o minúsculas. Por ejemplo, una etiqueta `<title>` podría escribirse como `<title>` , `<TITLE>` , `<Title>` , `<TiTlE>` , etc., y funcionará. Sin embargo, es una buena práctica escribir todas las etiquetas en minúsculas para mayor coherencia y legibilidad.

`<title>`

`<title>`

`<TITLE>`

`<Title>`

`<TiTlE>`

Anatomía de un elemento HTML

Exploremos más a fondo nuestro elemento de párrafo de la sección anterior:

La anatomía de nuestro elemento es:

- La etiqueta de apertura: Consiste en el nombre del elemento (en este ejemplo, p para el párrafo), envuelto en corchetes angulares de apertura y cierre. Esta etiqueta de apertura marca dónde inicia o comienza a surtir efecto el elemento. En este ejemplo, precede al inicio del texto del párrafo.
- El contenido: Este es el contenido del elemento. En este ejemplo, es el texto del párrafo.
- La etiqueta de cierre: Esta es la misma que la etiqueta de apertura, excepto que incluye una barra diagonal delante del nombre del elemento. Esto marca donde termina el elemento. No incluir una etiqueta de cierre es un error común para principiantes que puede producir resultados peculiares.

El elemento es la etiqueta de apertura, seguida del contenido, seguida de la etiqueta de cierre.

Aprendizaje activo: crear tu primer elemento HTML

Edite la siguiente línea en el área "Código editable" envolviéndola con las etiquetas `<em>` y `</em>`. Para abrir el elemento , coloque la etiqueta de apertura `<em>` al comienzo de la línea. Para cerrar el elemento , coloque la etiqueta de cierre `</em>` al final de la línea. ¡Hacer esto debería darle formato de texto en cursiva a la línea! Consulta la actualización de tus cambios en vivo en el área de Salida en vivo .

`<em>`

`</em>`.

`<em>`

`</em>`

Si comete un error, puede borrar su trabajo utilizando el botón Restablecer . Si te quedas realmente atascado, pulsa el botón Mostrar solución para ver la respuesta.

Salida en vivo

Código editable

Pulse Esc para alejar el foco del área de código (Tab inserta un carácter de tabulación).

Este es mi texto.

```
html {
  font-family: "Open Sans Light", Helvetica, Arial, sans-serif;
}
```

```
h2 {
  font-size: 16px;
}
```

```
.a11y-label {
  margin: 0;
  text-align: right;
  font-size: 0.7rem;
  width: 98%;
}
```

```
body {
  margin: 10px;
  background: #f5f9fa;
}
```

```
const textarea = document.getElementById("code");
const reset = document.getElementById("reset");
```

```

const solution = document.getElementById("solution");
const output = document.querySelector(".output");
const code = textarea.value;
let userEntry = textarea.value;

function updateCode() {
    output.innerHTML = textarea.value;
}

const htmlSolution = "Este es mi texto.";
let solutionEntry = htmlSolution;

reset.addEventListener("click", () => {
    textarea.value = code;
    userEntry = textarea.value;
    solutionEntry = htmlSolution;
    solution.value = "Mostrar solución";
    updateCode();
});

solution.addEventListener("click", () => {
    if (solution.value === "Mostrar solución") {
        textarea.value = solutionEntry;
        solution.value = "Ocultar solución";
    } else {
        textarea.value = userEntry;
        solution.value = "Mostrar solución";
    }
    updateCode();
});

textarea.addEventListener("input", updateCode);
window.addEventListener("load", updateCode);

// detener la tabulación de la tecla tabulación fuera del área de texto y
// hacer que escriba una tabulación en la posición del cursor en su lugar
textarea.onkeydown = (e) => {
    if (e.keyCode === 9) {
        e.preventDefault();
        insertAtCaret("\t");
    }

    if (e.keyCode === 27) {
        textarea.blur();
    }
};

function insertAtCaret(text) {
    const scrollPos = textarea.scrollTop;
    let caretPos = textarea.selectionStart;

    const front = textarea.value.substring(0, caretPos);
    const back = textarea.value.substring(
        textarea.selectionEnd,
        textarea.value.length,
    );
    textarea.value = front + text + back;
    caretPos += text.length;
    textarea.selectionStart = caretPos;
    textarea.selectionEnd = caretPos;
    textarea.focus();
    textarea.scrollTop = scrollPos;
}

// Actualizar el userCode guardado cada vez que el usuario actualice el código de área de texto
textarea.onkeyup = () => {
    // Solo queremos guardar el estado cuando se muestra el código de usuario,
    // no la solución, para que la solución no se guarde sobre el código de usuario
    if (solution.value === "Mostrar solución") {
        userEntry = textarea.value;
    } else {

```

```
        solutionEntry = textarea.value;
    }

    updateCode();
};
```

Anidando elementos

Los elementos se pueden colocar dentro de otros elementos. Esto se llama anidamiento . Si quisiéramos decir que nuestro gato está muy gruñón, podríamos envolver la palabra muy en un elemento `<strong>` , lo que significa que la palabra debe tener un formato de texto fuerte:

```
<strong>
    Mi gato es muy gruñón.
```

Hay una forma correcta e incorrecta de anidar. En el ejemplo anterior, abrimos primero el elemento `p` , luego abrimos el elemento `strong` . Para una anidación adecuada, primero debemos cerrar el elemento `strong` , antes de cerrar `p` .

```
p
strong
strong
p
```

El siguiente es un ejemplo de la forma incorrecta de anidar:

```
    Mi gato es muy gruñón.
```

Las etiquetas tienen que abrirse y cerrarse de manera que estén dentro o fuera la una de la otra . Con el tipo de superposición en el ejemplo anterior, el navegador tiene que adivinar tu intención. Este tipo de adivinanzas puede dar lugar a resultados inesperados.

Elementos vacíos

No todos los elementos siguen el patrón de una etiqueta de apertura, contenido y una etiqueta de cierre. Algunos elementos consisten en una sola etiqueta, que normalmente se utiliza para insertar/incrustar algo en el documento. Dichos elementos se denominan elementos vacíos . Por ejemplo, el elemento `<img>` incrusta un archivo de imagen en una página:

```
<img>
    
```

Esto daría como resultado lo siguiente:

Nota: En HTML, no es necesario añadir un `/` al final de la etiqueta de un elemento vacío, por ejemplo: `` . Sin embargo, también es una sintaxis válida, y puede hacerlo cuando desee que su HTML sea XML válido.

```
/
```

```

```

Atributos

Los elementos también pueden tener atributos. Los atributos se ven así:

Los atributos contienen información adicional sobre el elemento que no aparecerá en el contenido. En este ejemplo, el atributo `class` es un nombre de identificación utilizado para identificar el elemento con información de estilo.

`class`

Un atributo debe tener:

- Un espacio entre él y el nombre del elemento. (Para un elemento con más de un atributo, los atributos también deben estar separados por espacios).
- El nombre del atributo, seguido de un signo igual.
- Un valor de atributo, envuelto con comillas de apertura y cierre.

Aprendizaje activo: añadir atributos a un elemento

El elemento `<img>` puede tomar una serie de atributos, que incluyen:

`<img>`

`src`

El atributo `src` es un atributo requerido que especifica la ubicación de la imagen. Por ejemplo:
`src="https://raw.githubusercontent.com/mdn/beginner-html-site/gh-pages/images/firefox-icon.png"` .

`src`

`src="https://raw.githubusercontent.com/mdn/beginner-html-site/gh-pages/images/firefox-icon.png"`

`alt`

El atributo `alt` especifica una descripción de texto de la imagen. Por ejemplo: `alt="El icono de Firefox"` .

`alt`

`alt="El icono de Firefox"`

`width`

El atributo `width` especifica el ancho de la imagen con la unidad en píxeles. Por ejemplo: `width="300"` .

`width`

`width="300"`

`height`

El atributo `height` especifica la altura de la imagen con la unidad en píxeles. Por ejemplo: `height="300"` .

`height`

height="300"

Edita la siguiente línea en el área del Input para convertirla en una imagen.

- Encuentra tu imagen favorita en línea, haz clic con el botón derecho y pulsa Copiar enlace/dirección de imagen .

- De vuelta en el área de abajo, añade el atributo src y rellénalo con el enlace del paso 1.

src

- Establece el atributo alt .

alt

- Añade los atributos width y height .

width

height

Podrás ver los cambios en vivo en el área de Salida .

Si cometes un error, siempre puedes restablecerlo utilizando el botón Restablecer . Si te quedas realmente atascado, pulsa el botón Mostrar solución para ver la respuesta.

Salida en vivo

Código editable

Pulse Esc para alejar el foco del área de código (Tab inserta un carácter de tabulación).

```
html {
  font-family: sans-serif;
}

h2 {
  font-size: 16px;
}

.a11y-label {
  margin: 0;
  text-align: right;
  font-size: 0.7rem;
  width: 98%;
}

body {
  margin: 10px;
  background: #f5f9fa;
}

const textarea = document.getElementById("code");
const reset = document.getElementById("reset");
const solution = document.getElementById("solution");
const output = document.querySelector(".output");
const code = textarea.value;
let userEntry = textarea.value;

function updateCode() {
  output.innerHTML = textarea.value;
}

const htmlSolution =
  '';
let solutionEntry = htmlSolution;
```

```

reset.addEventListener("click", () => {
  textarea.value = code;
  userEntry = textarea.value;
  solutionEntry = htmlSolution;
  solution.value = "Mostrar solución";
  updateCode();
});

solution.addEventListener("click", () => {
  if (solution.value === "Mostrar solución") {
    textarea.value = solutionEntry;
    solution.value = "Ocultar solución";
  } else {
    textarea.value = userEntry;
    solution.value = "Mostrar solución";
  }
  updateCode();
});

textarea.addEventListener("input", updateCode);
window.addEventListener("load", updateCode);

// detener la tabulación de la tecla tabulación fuera del área de texto y
// hacer que escriba una tabulación en la posición del cursor en su lugar

textarea.onkeydown = (e) => {
  if (e.keyCode === 9) {
    e.preventDefault();
    insertAtCaret("\t");
  }

  if (e.keyCode === 27) {
    textarea.blur();
  }
};

function insertAtCaret(text) {
  const scrollPos = textarea.scrollTop;
  let caretPos = textarea.selectionStart;

  const front = textarea.value.substring(0, caretPos);
  const back = textarea.value.substring(
    textarea.selectionEnd,
    textarea.value.length,
  );
  textarea.value = front + text + back;
  caretPos += text.length;
  textarea.selectionStart = caretPos;
  textarea.selectionEnd = caretPos;
  textarea.focus();
  textarea.scrollTop = scrollPos;
}

// Actualizar el userCode guardado cada vez que el usuario actualice el código de área de texto

textarea.onkeyup = () => {
  // Solo queremos guardar el estado cuando se muestra el código de usuario,
  // no la solución, para que la solución no se guarde sobre el código de usuario
  if (solution.value === "Mostrar solución") {
    userEntry = textarea.value;
  } else {
    solutionEntry = textarea.value;
  }

  updateCode();
};

```

Atributos booleanos

A veces verás atributos escritos sin valores. Esto es totalmente aceptable. Estos se llaman atributos booleanos. Los atributos booleanos solo pueden tener un valor, que generalmente es el mismo que el nombre del atributo. Por ejemplo, considere el atributo `disabled`, que puede asignar a los elementos de entrada de formulario. (Utiliza esto para deshabilitar los elementos de entrada del formulario para que el usuario no pueda realizar entradas. Los elementos deshabilitados suelen tener un aspecto grisáceo.) Por ejemplo:

```
disabled
```

Como abreviatura, es aceptable escribir esto de la siguiente manera:

Como referencia, el ejemplo anterior también incluye un elemento de entrada de formulario no deshabilitado. El HTML del ejemplo anterior produce este resultado:

Omitir comillas en torno a los valores de atributo

Si observa el código de muchos otros sitios, es posible que encuentre una serie de estilos de marcado extraños, incluidos los valores de atributos sin comillas. Esto está permitido en ciertas circunstancias, pero también puede romper su margen de beneficio en otras circunstancias. El elemento en el fragmento de código a continuación, `<a>`, se llama ancla. Los anclajes encierran texto y los convierten en enlaces. El atributo `href` especifica la dirección web a la que apunta el enlace. Puedes escribir esta versión básica a continuación con solo el atributo `href`, así:

```
<a>
href
href
    sitio web favorito
```

Los anclajes también pueden tener un atributo `title`, una descripción de la página vinculada. Sin embargo, tan pronto como añadimos el `title` de la misma manera que el atributo `href` hay problemas:

```
title
title
href
    sitio web favorito
```

Como se ha escrito anteriormente, el navegador malinterpreta el marcado, confundiendo el atributo `title` con tres atributos: un atributo `title` con el valor `The`, y dos atributos booleanos, `Mozilla` y `homepage`. ¡Obviamente, esto no es la intención! Causará errores o comportamientos inesperados, como puede ver en el ejemplo en vivo a continuación. ¡Intenta pasar el cursor sobre el enlace para ver el texto del título!

```
title
The
Mozilla
homepage
```

Incluya siempre las comillas de los atributos. Evita tales problemas y da como resultado un código más legible.

¿Comillas simples o dobles?

En este artículo, también notarás que los atributos están entre comillas dobles. Sin embargo, es posible que veas comillas simples en algún código HTML. Es una cuestión de estilo. Puedes elegir libremente cuál prefieres. Ambas opciones son equivalentes:

```
Un enlace a mi ejemplo.
```

```
Un enlace a mi ejemplo.
```

Asegúrate de no mezclar comillas simples y comillas dobles. Este ejemplo (a continuación) muestra una especie de mezcla de comillas que saldrá mal:

```
<a href="https://www.example.com">Un enlace a mi ejemplo.
```

Sin embargo, si utilizas un tipo de comilla, puedes incluir el otro tipo de comilla dentro tus valores de atributo:

```
Un enlace a mi ejemplo.
```

Para usar comillas dentro de otras comillas del mismo tipo (comillas simples o comillas dobles), use entidades HTML . Por ejemplo, esto se romperá:

```
Un enlace a mi ejemplo.
```

En su lugar, debe hacer lo siguiente:

```
Un enlace a mi ejemplo.
```

Anatomía de un documento HTML

Los elementos HTML individualmente no son muy útiles por sí solos. A continuación, examinemos cómo se combinan los elementos para formar una página HTML completa:

```
Mi página de prueba
```

```
Esta es mi página
```

Aquí tenemos:

- `<!DOCTYPE html>` : El tipo de documento. Cuando HTML era joven (1991-1992), los doctype estaban destinados a actuar como enlaces a un conjunto de reglas que la página HTML tenía que seguir para ser considerada un buen HTML. Los doctypes solían parecerse a algo como esto: `html <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">` Más recientemente, el doctype es un artefacto histórico que debe incluirse para que todo lo demás funcione correctamente. `<!DOCTYPE html>` es la cadena más corta de caracteres que cuenta como un tipo de documento válido. ¡Eso es todo lo que necesitas saber!

`<!DOCTYPE html>` : El tipo de documento. Cuando HTML era joven (1991-1992), los doctype estaban destinados a actuar como enlaces a un conjunto de reglas que la página HTML tenía que seguir para ser considerada un buen HTML. Los doctypes solían parecerse a algo como esto:

```
<!DOCTYPE html>
```

Más recientemente, el doctype es un artefacto histórico que debe incluirse para que todo lo demás funcione correctamente. `<!DOCTYPE html>` es la cadena más corta de caracteres que cuenta como un tipo de documento válido. ¡Eso es todo lo que necesitas saber!

```
<!DOCTYPE html>
```

- `<html></html>` : El elemento `<html>` . Este elemento envuelve todo el contenido de la página. A veces se le conoce como el elemento raíz.

`<html></html>` : El elemento `<html>` . Este elemento envuelve todo el contenido de la página. A veces se le conoce como el elemento raíz.

```
<html></html>
```

```
<html>
```

- `<head></head>` : El elemento `<head>` . Este elemento actúa como un contenedor para todo lo que desea incluir en la página HTML, que no es el contenido que la página mostrará a los espectadores. Esto incluye palabras clave y una descripción de la página que aparecería en los resultados de búsqueda, CSS para dar estilo al contenido, declaraciones de conjuntos de caracteres y más. Aprenderás más sobre esto en el próximo artículo de la serie.

`<head></head>` : El elemento `<head>` . Este elemento actúa como un contenedor para todo lo que desea incluir en la página HTML, que no es el contenido que la página mostrará a los espectadores. Esto incluye palabras clave y una descripción de la página que aparecería en los resultados de búsqueda, CSS para dar estilo al contenido, declaraciones de conjuntos de caracteres y más. Aprenderás más sobre esto en el próximo artículo de la serie.

```
<head></head>
```

```
<head>
```

- `<meta charset="utf-8">` : El elemento `<meta>` . Este elemento representa metadatos que no pueden ser representados por otros elementos HTML relacionados con metadatos, como `<base>` , `<link>` , `<script>` , `<style>` o `<title>` . El atributo `charset` especifica la codificación de caracteres para su documento como UTF-8, que incluye la mayoría de los caracteres de la gran mayoría de los lenguajes humanos escritos. Con esta configuración, la página ahora puede manejar cualquier contenido textual que pueda contener. No hay razón para no configurar esto, y puede ayudar a evitar algunos problemas más adelante.

`<meta charset="utf-8">` : El elemento `<meta>` . Este elemento representa metadatos que no pueden ser representados por otros elementos HTML relacionados con metadatos, como `<base>` , `<link>` , `<script>` , `<style>` o `<title>` . El atributo `charset` especifica la codificación de caracteres para su documento como UTF-8, que incluye la mayoría de los caracteres de la gran mayoría de los lenguajes humanos escritos. Con esta configuración, la página ahora puede manejar cualquier contenido textual que pueda contener. No hay razón para no configurar esto, y puede ayudar a evitar algunos problemas más adelante.

```
<meta charset="utf-8">
```

```
<meta>
```

```
<base>
```

```
<link>
```

```
<script>
```

```
<style>
```

```
<title>
```

```
charset
```

- `<title></title>` : El elemento `<title>` . Esto establece el título de la página, que es el título que aparece en la pestaña del navegador en la que se carga la página. El título de la página también se utiliza para describir la página cuando está en marcadores.

`<title></title>` : El elemento `<title>` . Esto establece el título de la página, que es el título que aparece en la pestaña del navegador en la que se carga la página. El título de la página también se utiliza para describir la página cuando está en marcadores.

```
<title></title>
```

```
<title>
```

- `<body></body>` : El elemento `<body>` . Contiene todo el contenido que se muestra en la página, incluidos texto, imágenes, vídeos, juegos, pistas de audio reproducibles o cualquier otra cosa.

`<body></body>` : El elemento `<body>` . Contiene todo el contenido que se muestra en la página, incluidos texto, imágenes, vídeos, juegos, pistas de audio reproducibles o cualquier otra cosa.

```
<body></body>
```

```
<body>
```

Aprendizaje activo: añadir algunas funciones a un documento HTML

Si quieres experimentar con escribir algo de HTML en tu ordenador local, puedes:

- Copie el ejemplo de página HTML listado anteriormente.
- Crea un nuevo archivo en tu editor de texto.
- Pega el código en el nuevo archivo de texto.
- Guarde el archivo como `index.html` .

```
index.html
```

Nota: También puedes encontrar esta plantilla HTML básica en el repositorio de GitHub del área de aprendizaje de MDN .

Ahora puede abrir este archivo en un navegador web para ver cómo se ve el código renderizado. Edita el código y actualiza el navegador para ver cuál es el resultado. Inicialmente, la página se ve así:

En este ejercicio, puedes editar el código localmente en tu ordenador, como se describió anteriormente, o puedes editarlo en la ventana de muestra a continuación (la ventana de muestra editable representa solo el contenido del elemento `<body>` , en este caso). Mejora tus habilidades implementando las siguientes tareas:

```
<body>
```

- Justo debajo de la etiqueta de apertura del elemento `<body>` , añade un título principal para el documento. Esto debe envolverse dentro de una etiqueta `<h1>` de apertura y `</h1>` de cierre.

```
<body>
```

```
<h1>
```

```
</h1>
```

- Edita el contenido del párrafo para incluir texto sobre un tema que te parezca interesante.

- Haz que las palabras importantes se destaquen en negrita envolviéndolas dentro de una etiqueta `<strong>` de apertura y `</strong>` de cierre.

- Añade un enlace a tu párrafo, como se explicó anteriormente en el artículo .

- Añade una imagen a tu documento. Colócalo debajo del párrafo, como se explicó anteriormente en el artículo . Gana puntos de bonificación si consigues vincular a una imagen diferente (ya sea localmente en tu ordenador o en algún otro lugar de la web).

Si comete un error, siempre puede restablecerlo utilizando el botón Restablecer . Si te quedas realmente atascado, pulsa el botón Mostrar solución para ver la respuesta.

Salida en vivo

Código editable

Pulse Esc para alejar el foco del área de código (Tab inserta un carácter de tabulación).

Esta es mi página

```
html {  
  font-family: sans-serif;  
}
```

```
h1 {  
  color: blue;  
}
```

```
h2 {  
  font-size: 16px;  
}
```

```
.a11y-label {  
  margin: 0;  
  text-align: right;  
  font-size: 0.7rem;  
  width: 98%;  
}
```

```
img {  
  max-width: 100%;  
}
```

```
body {  
  margin: 10px;  
  background: #f5f9fa;  
}
```

```
const textarea = document.getElementById("code");  
const reset = document.getElementById("reset");  
const solution = document.getElementById("solution");  
const output = document.querySelector(".output");  
const code = textarea.value;  
let userEntry = textarea.value;
```

```
function updateCode() {  
  output.innerHTML = textarea.value;  
}
```

```
const htmlSolution =  
  'Algo de músicaDisfruto mucho tocando la batería. Uno de mis bateristas favoritos es Neal Peart, que to  
let solutionEntry = htmlSolution;
```

```
reset.addEventListener("click", () => {  
  textarea.value = code;  
  userEntry = textarea.value;  
  solutionEntry = htmlSolution;  
  solution.value = "Mostrar solución";  
});
```

```

        updateCode();
    });

    solution.addEventListener("click", () => {
        if (solution.value === "Mostrar solución") {
            textarea.value = solutionEntry;
            solution.value = "Ocultar solución";
        } else {
            textarea.value = userEntry;
            solution.value = "Mostrar solución";
        }
        updateCode();
    });

    textarea.addEventListener("input", updateCode);
    window.addEventListener("load", updateCode);

    // detener la tabulación de la tecla tabulación fuera del área de texto y
    // hacer que escriba una tabulación en la posición del cursor en su lugar

    textarea.onkeydown = (e) => {
        if (e.keyCode === 9) {
            e.preventDefault();
            insertAtCaret("\t");
        }

        if (e.keyCode === 27) {
            textarea.blur();
        }
    };

    function insertAtCaret(text) {
        const scrollPos = textarea.scrollTop;
        let caretPos = textarea.selectionStart;

        const front = textarea.value.substring(0, caretPos);
        const back = textarea.value.substring(
            textarea.selectionEnd,
            textarea.value.length,
        );
        textarea.value = front + text + back;
        caretPos += text.length;
        textarea.selectionStart = caretPos;
        textarea.selectionEnd = caretPos;
        textarea.focus();
        textarea.scrollTop = scrollPos;
    }

    // Actualizar el userCode guardado cada vez que el usuario actualice el código de área de texto
    textarea.onkeyup = () => {
        // Solo queremos guardar el estado cuando se muestra el código de usuario,
        // no la solución, para que la solución no se guarde sobre el código de usuario
        if (solution.value === "Mostrar solución") {
            userEntry = textarea.value;
        } else {
            solutionEntry = textarea.value;
        }

        updateCode();
    };

```

Espacios en blanco en HTML

En los ejemplos anteriores, es posible que hayas notado que se incluyen muchos espacios en blanco en el código. Este paso es opcional. Estos dos fragmentos de código son equivalentes:

Los perros son tontos.

Los perros

```
son
  tontos.
```

No importa cuánto espacio en blanco utilice dentro del contenido del elemento HTML (que puede incluir uno o más caracteres de espacio, o también saltos de línea), el analizador HTML reduce cada secuencia de espacio en blanco a un solo espacio al representar el código. Entonces, ¿por qué usar tanto espacio en blanco? La respuesta es la legibilidad.

Puede ser más fácil entender lo que está pasando en tu código si lo tienes bien formateado. En nuestro HTML tenemos cada elemento anidado indentado por dos espacios más que el que está dentro. Depende de usted elegir el estilo de formato (cuántos espacios para cada nivel de indentación, por ejemplo), pero debe considerar formatearlo.

Echemos un vistazo a cómo el navegador representa los dos párrafos anteriores con y sin espacios en blanco:

Nota: El acceso a `innerHTML` de los elementos de JavaScript mantendrá intacto todo el espacio en blanco.

Esto puede arrojar resultados inesperados si el navegador recorta el espacio en blanco.

```
const noWhitespace = document.getElementById("noWhitespace").innerHTML;
console.log(noWhitespace);
// "Los perros son tontos".

const whitespace = document.getElementById("whitespace").innerHTML;
console.log(whitespace);
// "Los perros
//    son
//      tontos."
```

Referencias de entidades: Incluir caracteres especiales en HTML

En HTML, los caracteres `<`, `>`, `"`, `'` y `&` son caracteres especiales. Son partes de la sintaxis HTML en sí. Entonces, ¿cómo incluyes uno de estos caracteres especiales en tu texto? Por ejemplo, si desea usar un signo ampersand (&) o menor que, y no interpretarlo como código.

`<`

`>`

`"`

`'`

`&`

Haces esto con referencias de caracteres. Estos son códigos especiales que representan caracteres, para ser utilizados en estas circunstancias exactas. Cada referencia de carácter comienza con un ampersand (&) y termina con un punto y coma (;).

`<`

`>`

`"`

`'`

`&`

El equivalente de referencia de carácter podría recordarse fácilmente porque el texto que utiliza puede verse como menor que (less than, en inglés) para '<', comilla (quot, en inglés) para '"' y de manera similar para otros. Para obtener más información sobre las referencias de entidades, consulta Lista de referencias de entidades de caracteres XML y HTML (Wikipedia).

En el siguiente ejemplo, hay dos párrafos:

```
En HTML, se define un párrafo utilizando el elemento .
```

```
En HTML, se define un párrafo utilizando el elemento .
```

En la salida en vivo a continuación, puede ver que el primer párrafo ha salido mal. El navegador interpreta la segunda instancia de `<p>` como comenzar un nuevo párrafo. El segundo párrafo se ve bien porque tiene corchetes angulares con referencias de caracteres.

```
<p>
```

Nota: No es necesario usar referencias de entidad para ningún otro símbolo, ya que los navegadores modernos manejarán bien los símbolos reales siempre que la codificación de caracteres de tu HTML esté configurada en UTF-8 .

Comentarios en HTML

HTML tiene un mecanismo para escribir comentarios en el código. Los navegadores ignoran los comentarios, lo que hace que los comentarios sean invisibles para el usuario. El propósito de los comentarios es permitirle incluir notas en el código para explicar su lógica o codificación. Esto es muy útil si vuelve a una base de código después de estar ausente el tiempo suficiente para no recordarlo por completo. Del mismo modo, los comentarios son invaluable ya que diferentes personas están haciendo cambios y actualizaciones.

Para escribir un comentario en HTML, envuélvalo en los marcadores especiales `<!--` y `-->` . Por ejemplo:

```
<!--
```

```
-->
```

```
No estoy dentro de un comentario
```

Como puede ver a continuación, solo se muestra el primer párrafo en la salida en vivo.

Resumen

¡Has llegado al final del artículo! Esperamos que hayas disfrutado de tu recorrido por los conceptos básicos de HTML.

En este punto, debes entender cómo se ve HTML y cómo funciona a un nivel básico. También deberías poder escribir algunos elementos y atributos. Los artículos posteriores de este módulo profundizan en algunos de los temas introducidos aquí, así como en la presentación de otros conceptos del lenguaje.

- A medida que comiences a aprender más sobre HTML, considera aprender los conceptos básicos de CSS (hojas de estilo en cascada). CSS es el lenguaje utilizado para dar estilo a las páginas web, como cambiar fuentes o colores o alterar el diseño de la página. HTML y CSS funcionan bien juntos, como pronto descubrirás.

Vease también

- Aplicar color a elementos HTML usando CSS

- Overview: Estructurando la web con HTML

- Siguiente

Help improve MDN

This page was last modified on 12 sept 2026 by MDN contributors .